

HaskellKatas

Learning by Training Your Eye



```
stack ghci
diff --git a/test/MultiplySpec.hs b/test/MultiplySpec.hs
index 24d0cc9..aea583a 100644
--- a/test/MultiplySpec.hs
+++ b/test/MultiplySpec.hs
@@ -12,7 +12,7 @@ spec :: Spec
spec = do
  describe "Basic tests" $ do
    it "multiply 7 7 == 49" $
      multiply 7 7 `shouldBe` 49 -- wip 3
    it "multiply 7 7 'shouldBe' 49 -- wip 4
    it "multiply 11 11 == 121" $
      multiply 11 11 `shouldBe` 121
  describe "Randomized Tests" $ do
    property $ \ x y -> multiply x y `shouldBe` (x * y)

Setting up watches.
Watches established.

stack build
/multiply-test ...
multiply: copy/register
Installing library in /home/naldoco/haskellkatas/multiply/.stack-work/l
install/x86_64-linux/aea5a56403c5ddb65598531c319417732e05414e8b0bb1a98d0
eedb025b45fa2/8.6.5/lib/x86_64-linux-ghc-8.6.5/multiply-0.1.0.0-3HFxcwI
XSRc9T5dzuZnbnm
Registering library for multiply-0.1.0.0..
multiply> test (suite: multiply-test)

Progress 1/2: multiply
Multiply
  Basic tests
    multiply 7 7 == 49
    multiply 11 11 == 121
  Randomized Tests
    ...arbitrary numbers...
    +++ OK, passed 100 tests.

Finished in 0.0021 seconds
0 examples, 0 failures

multiply: Test suite multiply-test passed
Completed 2 action(s).
Success! Waiting for next file change.
Type help for available commands. Press enter to force a rebuild.
```



ZuriHac 2023

Reynaldo Cordero @HaskellKatas
SS.II. Universidad de Alcalá // HaskellMAD



HaskellKatas

- Programming: teaching **thinking**
- Real “**Scientific Method** Experience”
- **Advanced Mathematics** in practice
- Squeeze **expressiveness** in language
- Prioritize the ability to **observe**
- Effective **comment writing**
- Care and importance of **details**
- **Artistic** sense and **style**
- Unix Philosophy: **Minimalist, Modular, Hackable**
- 100% Free software (**FLOSS**)
 - <https://gitlab.com/HaskellKatas/katas--proof-of-concept>

Serendipity

Serendipity is an **unplanned** lucky **discovery**. Serendipity is a common occurrence throughout the history of product invention and scientific discovery. Serendipity is also seen as a potential design principle for online activities that would present a wide range of information and viewpoints, **rather than just reinforcing a user's opinion**. (Wikipedia)

The key to serendipity is to privilege **observation**.

Katafication

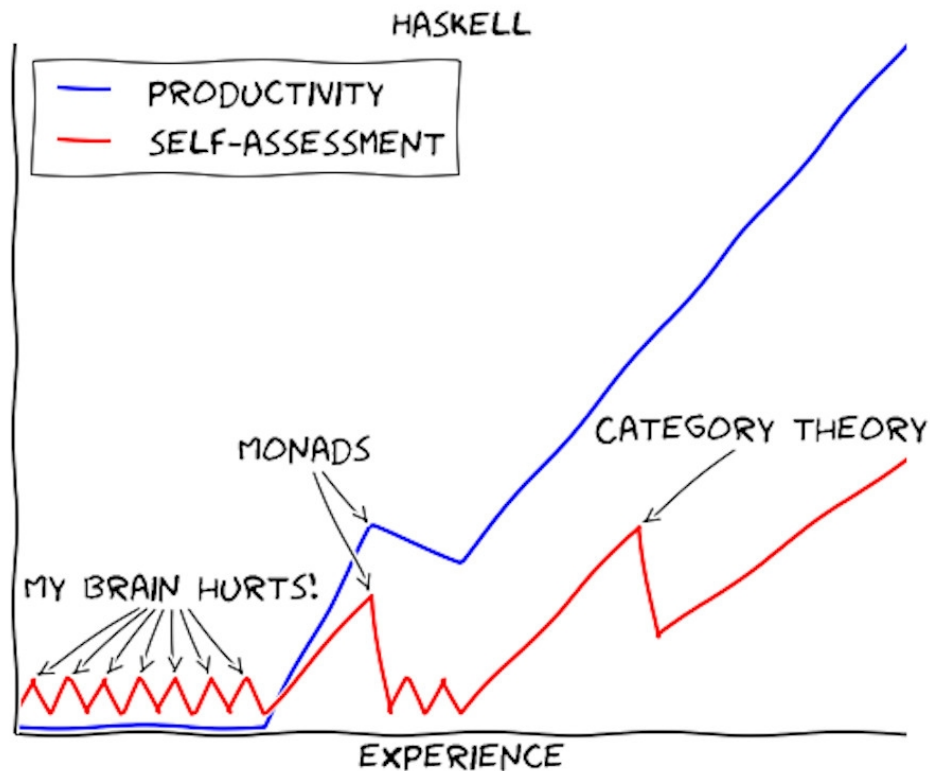
Code katafication consists of fast **reformatting** and **refactoring** computer code written in **Haskell** ^(*), both your own and that of other programmers, to find your **own style** that brings out the structures and makes them easily **relatable**, stimulating **experimentation**.

(*) A **general-purpose**, **concise**, **clean**, **easy-to-read** (minimum boilerplate) **purely functional** programming language designed for **teaching**, **research** and **industrial** applications. **statically-typed**, **type inference**, **referential transparency**, **lazy evaluation** ... (You need all of that to **keep it simple**, **mathematical**, and **reduce error categories** to a minimum).

Studying vs Katafication-Gym

LEARNING CURVE "STUDYING APPROACH"

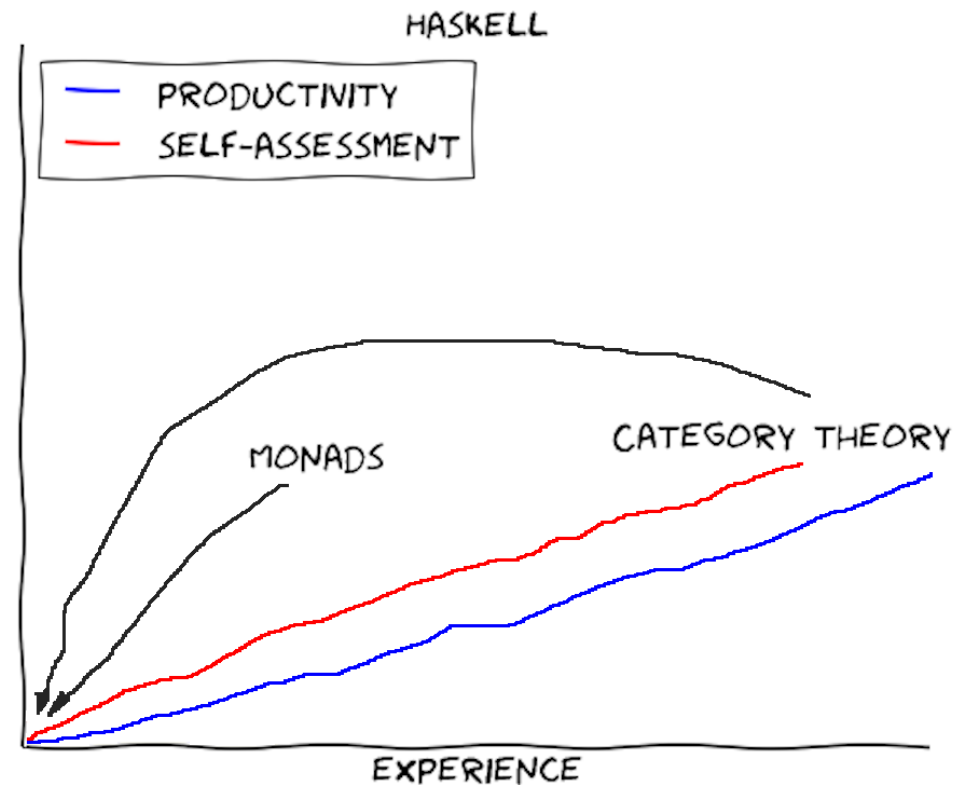
BOOKS / TUTORIALS / EXERCISES / TEACHERS



https://github.com/Dobiasd/articles/blob/master/programming_language_learning_curves.md

LEARNING CURVE "KATAFICATION-GYM"

EXPRESSIVE-ARTISTIC FORMATTING / FAST REFACTORING
EXPERIMENTS ON THE FLY



Stealed and changed

Enjoy!



Katas -proof of concept

<https://gitlab.com/HaskellKatas/katas--proof-of-concept>

Recomended Kata Library: Haskell Practice

<https://www.codewars.com/kata/search/haskell>



Kata Training: Multiply

<https://www.codewars.com/kata/50654ddff44f800200000004/train/haskell>